

Trainer Guide: Spring Boot Session 4 - Hands-on Lab

A walk-through script for [springboot-session4-lab.html](#) - standing up a Eureka registry, two microservices, multiple instances, and an API gateway.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session4-lab.html](#)

Each lab has an ON SCREEN reminder and a SAY / WALK THROUGH section you can read out as you lead. EMPHASISE and DELIVERY TIP notes are for you only.

Lab flow, the architecture map & RAM warning

ON SCREEN

The lab page: a pinned architecture map (Client to Gateway to Eureka to the services), a 9-step checklist, and the version-compatibility warning box.

SAY / WALK THROUGH

This lab is different from every previous one. Instead of one project, you'll run four separate Spring Boot apps at once - then six, once we scale Student MS. Keep the architecture map pinned at the top in view the whole time: client to gateway, gateway to Eureka to find services, then to the right instance. Two warnings up front. One: running four-to-six JVMs needs real RAM - if a machine struggles, stop the extra Student MS instances after they've registered. Two: the start order matters - always start Eureka first, then the microservices, then the gateway.

EMPHASISE

Read the version-compatibility box aloud and have everyone check it - a Spring Boot and Spring Cloud version mismatch is the single biggest cause of failure in this session. `start.spring.io` pairs them correctly; if anyone sets versions by hand, point them at that table.

Pre-requisites & ports

ON SCREEN

Pre-requisites - Java 17+, four IDE windows, 8 GB RAM - and the port map: 8761, 8080, 8081, 8082, 8083, 9393.

SAY / WALK THROUGH

Pre-requisites: Java 17 or higher, and ideally four IDE windows since these are four independent projects. Postman or a browser for testing. And at least 8 gigabytes of free RAM. Memorise the port map, because we'll refer to it constantly: Eureka on 8761, Student MS on 8080 - plus 8082 and 8083 for extra instances, Customer MS on 8081, and the API Gateway on 9393. If you keep those straight, the whole lab stays clear.

DELIVERY TIP

Have Eureka pre-running on your own machine before the session. It's your safety net - if a learner's setup stalls, your live demo never breaks and the group keeps moving.

Part 1 - Eureka & the microservices

Lab 1 - Create the Eureka Server

ON SCREEN

Steps to create the eureka-server project with one dependency, add `@EnableEurekaServer`, configure the four properties, and open localhost:8761.

SAY / WALK THROUGH

Lab one - the registry, the phonebook every service registers with. From start dot spring dot io create a project called eureka-server with one dependency: Eureka Server. On the main class, add `@EnableEurekaServer` - that single annotation turns this app into the registry. Then four properties: port 8761, an app name, and the two important falses - `register-with-eureka` false and `fetch-registry` false. Explain those: Eureka components are clients by default; for the server we switch both off because it shouldn't register with itself and it doesn't fetch a registry - it IS the registry. Run it and open localhost colon 8761. You'll see the Eureka dashboard with 'no instances available' - correct, because nothing has registered yet.

Lab 2 - Create the Student Microservice

ON SCREEN

Steps to create student-ms with Web + Eureka Client, a `StudentController` returning the running port, and the three client properties. STUDENT-MS appears in Eureka.

SAY / WALK THROUGH

Lab two - our first microservice. Create student-ms with Spring Web, Eureka Discovery Client, and DevTools. Write a simple `StudentController` at slash students that returns a string including the running port - we inject `Environment` and read local dot server dot port, so the response literally tells us which instance answered. That detail is what lets us see load balancing later, so don't skip it. Then three properties: port 8080, `spring dot application dot name` equals STUDENT-MS in uppercase, and the `defaultZone` pointing at Eureka. Run it, refresh localhost 8761, and STUDENT-MS now appears in the instances table.

EMPHASISE

Stress the service name - it's the one line that breaks everything if wrong. `spring dot application dot name` equals STUDENT-MS is the exact name Eureka lists it under and the exact name the gateway will route to. A typo here means the gateway route silently fails to resolve.

Lab 3 - Create the Customer Microservice

ON SCREEN

Steps to create customer-ms the same way - port 8081, name CUSTOMER-MS, a /customers endpoint. Both services now show in Eureka.

SAY / WALK THROUGH

Lab three is the same pattern as lab two, so go fast - repetition is the lesson here, that every microservice registers the same way. Create customer-ms, with the same two dependencies. Write a CustomerController at slash customers returning its port. Set the three properties: port 8081, name CUSTOMER-MS, same defaultZone. Run it and refresh Eureka - now both STUDENT-MS and CUSTOMER-MS show as UP. Two independent services, each registered in the phonebook, neither knowing about the other. That independence is the whole microservices idea, made concrete.

Lab 4 - Run Multiple Instances of Student MS

ON SCREEN

IntelliJ and Eclipse steps to run the same project on ports 8082 and 8083 via a VM argument, plus a command-line option. Eureka shows STUDENT-MS with 3 instances.

SAY / WALK THROUGH

Lab four - the visual payoff: horizontal scaling on one laptop. We don't create new projects; we run the same student-ms again on different ports using a VM argument. In IntelliJ, copy the run configuration and add VM option dash-D-server-dot-port equals 8082, then another for 8083. In Eclipse, a Maven build with goal spring-boot-run and the same VM argument. There's also a command-line option on the sheet. Now refresh Eureka: the STUDENT-MS row shows three instances - 8080, 8082, and 8083. To Eureka and the gateway, three instances on three ports look exactly like three instances on three servers in production.

EMPHASISE

If all three show the same port, the VM argument didn't apply - check it's in VM options, not Program arguments, and starts with dash-D. This is the most common stumble in this lab.

Part 2 - The API Gateway

Lab 5 - Create the API Gateway

ON SCREEN

Steps to create api-gateway with Gateway + Eureka Discovery Client, the base properties on port 9393, and a note about the reactive web type.

SAY / WALK THROUGH

Lab five - the single front door. Create api-gateway with two dependencies: Gateway and Eureka Discovery Client. There's no controller - the gateway is configuration only. Set the base properties: port 9393, an app name, and the defaultZone pointing at Eureka. Run it and confirm API-GATEWAY appears in the Eureka dashboard alongside the other two. One gotcha to mention: Spring Cloud Gateway runs on the reactive Netty server, not Tomcat. If you accidentally also add spring-boot-starter-web, they conflict - so don't, and if you hit a web-application-type warning, add spring dot main dot web-application-type equals reactive.

Lab 6 - Manual Route Configuration

ON SCREEN

The two manual route blocks for STUDENT-MS and CUSTOMER-MS in application.properties, with callouts on lb:// and the path predicate.

SAY / WALK THROUGH

Lab six - teach the gateway to route. A route has three parts: an id - any name; a uri - which service to forward to; and a predicate - which incoming URLs trigger it. Add two routes in properties. Route zero: id student-ms-route, uri lb colon slash slash STUDENT-MS, predicate path equals slash students-api slash star-star. Route one: the same shape for CUSTOMER-MS and slash customers-api. Restart the gateway. Two things to explain. lb colon slash slash means load balancer - it tells the gateway, don't use a hardcoded address, ask Eureka for the live instances and load-balance across them. And the predicate means any request whose path starts with slash students-api gets forwarded to STUDENT-MS.

Lab 7 - Test Load Balancing (the payoff)

ON SCREEN

Postman GET to localhost:9393/students-api/students, hit repeatedly, with the response port cycling 8080, 8082, 8083. Customer MS tested too.

SAY / WALK THROUGH

Lab seven - the moment the whole session builds to. In Postman, GET localhost colon 9393 slash students-api slash students - through the gateway, on the single gateway port. Hit it three or four times rapidly and watch the port in the response string change each time: 8080, then 8082, then 8083, then back to 8080. That's round-robin load balancing, live - one URL, traffic spread automatically across three instances. Then test slash customers-api slash customers to confirm the gateway reaches Customer MS too.

EMPHASISE

Make everyone hit the endpoint several times and read the changing port aloud. If you can, tile the three Student MS consoles so they see which instance logs each request. This is the single most memorable demo of the session - don't rush it.

Lab 8 - Discovery Locator Approach

ON SCREEN

Steps to comment out the manual routes, enable discovery locator with two properties, restart, and test using the service name in the URL.

SAY / WALK THROUGH

Lab eight shows the alternative to manual routes. Comment out the two manual routes from lab six. Add two lines: `discovery dot locator dot enabled equals true`, and `lower-case-service-id equals true`. Restart the gateway. Now test with the service name in the path: `localhost 9393 slash student-ms slash students`, and `slash customer-ms slash customers`. Both work with no manual route config at all, and Student MS still round-robins across its instances. Then compare the two approaches out loud: manual routes give clean, custom URLs and full control - better for production. Discovery locator is automatic and zero-config but leaks the service name into the URL - better for quick development. The trade-off is URL cleanliness versus zero configuration.

Lab 9 - Bonus: Observe the Eureka Heartbeat

ON SCREEN

A bonus: stop one Student MS instance, wait, watch Eureka drop it from 3 to 2, and see the gateway stop routing to it.

SAY / WALK THROUGH

If there's time, this bonus shows self-healing. Stop one Student MS instance - say the 8082 one. Wait thirty to sixty seconds, because Eureka has a grace period before it evicts a missing instance. Refresh the dashboard: STUDENT-MS drops from three instances to two. Now hit the gateway endpoint repeatedly again - traffic only cycles between 8080 and 8083 now, with no errors. The lesson: Eureka detected the missing heartbeat and removed the dead instance, and the gateway adjusted automatically. No config change, no restart. In production, instances crash and scale down all the time - this heartbeat-driven eviction is what keeps traffic flowing only to healthy instances.

Reference & close

Complete configuration reference & series wrap-up

ON SCREEN

The complete `application.properties` for all four projects, the main-class snippets, and the quick-reference card.

SAY / WALK THROUGH

The reference at the bottom has every `application dot properties` file for all four projects, plus the two main-class snippets and a note that `at-EnableEurekaClient` is optional in modern Spring Cloud. There's a reminder too: always start Eureka first. And this is the end of the series, so make the wrap meaningful. Across four sessions you went from a single Hello World controller, to a REST API, to a layered application with a database, to - today - a running microservices system with a registry, a gateway, multiple instances, and load balancing. That's a genuine, production-shaped microservices skeleton. In the real world it grows with a config server, JWT security, Docker, Kubernetes, and tracing - but you now have the foundation every one of those builds on. Thank you, all of you, for completing the series.